

Parallel Image Convolution

Analysis Report

High Performance Computing (HPC)
7th Semester — BSc. Engineering

Implementations: Serial, OpenMP, POSIX Pthreads, MPI, CUDA, MPI+OpenMP
Filters Tested: Gaussian Blur, Edge Detection, Sharpen
Repository: Parallel-Image-Convolution
Date: May 2026

Team Members

Imalsha Ranasinghe

Contents

1	Introduction	2
2	Parallel Programming Concepts Applied	2
2.1	Overview of the Parallelisation Strategy	2
2.2	Serial Baseline	2
2.3	OpenMP — Shared-Memory Thread Parallelism	3
2.4	POSIX Pthreads — Explicit Thread Management	4
2.5	MPI — Distributed-Memory Parallelism	4
2.6	CUDA — GPU Massive Parallelism	5
2.7	Hybrid MPI + OpenMP — Two-Level Parallelism	6
2.8	Summary of Parallelisation Concepts	8
3	Accuracy Analysis	8
3.1	Metric: Root Mean Square Error (RMSE)	8
3.2	Expected Sources of Error by Implementation	8
3.3	RMSE Results	9
4	Performance Analysis	10
4.1	Experimental Setup	10
4.2	Gaussian Blur — Execution Time vs. Thread/Process Count	10
4.3	Speedup and Amdahl’s Law	12
4.4	Hybrid MPI+OpenMP Configurations	12
4.5	CUDA Performance	13
4.6	Edge Detection and Sharpen (3×3)	14
4.7	Memory Bandwidth Bottleneck	17
4.8	Parallel Efficiency	18
5	Conclusion	18

1 Introduction

Image convolution is a fundamental operation in computer vision and image processing. Given an input image I and a convolution kernel K of size $k \times k$, the output pixel at position (x, y) and colour channel c is:

$$O(x, y, c) = \sum_{ky=-\lfloor k/2 \rfloor}^{\lfloor k/2 \rfloor} \sum_{kx=-\lfloor k/2 \rfloor}^{\lfloor k/2 \rfloor} I(\text{clamp}(x + kx, 0, W-1), \text{clamp}(y + ky, 0, H-1), c) \cdot K(kx, ky) \quad (1)$$

For a $W \times H$ image with C channels, Eq. (1) requires $W \cdot H \cdot C \cdot k^2$ multiply-accumulate operations. For our Gaussian blur kernel ($k = 21$, $k^2 = 441$) on a 2000×1333 image with $C = 3$ channels this is $\approx 3.5 \times 10^9$ operations — clearly motivating parallelism.

This report describes five parallel implementations (OpenMP, POSIX Pthreads, MPI, CUDA, and a Hybrid MPI+OpenMP approach), analyses their correctness relative to the serial baseline using Root Mean Square Error (RMSE), and benchmarks execution time as a function of the degree of parallelism.

2 Parallel Programming Concepts Applied

2.1 Overview of the Parallelisation Strategy

All parallel implementations exploit the *embarrassingly parallel* nature of 2D convolution: each output pixel is computed independently, depending only on a fixed-size neighbourhood of *read-only* input pixels. No inter-pixel communication is needed during computation; only load balancing and result assembly require coordination.

The dominant decomposition is **row-block partitioning**: the image rows are divided into contiguous chunks and assigned to parallel workers (threads or processes), ensuring each worker accesses a contiguous memory region and maximising cache locality.

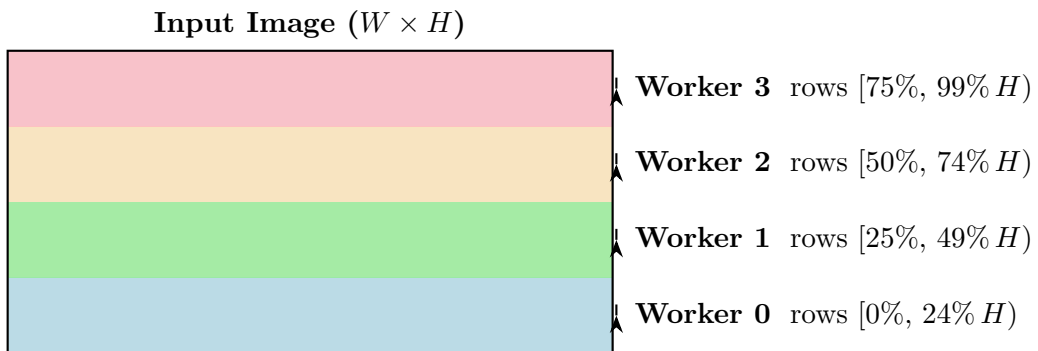


Figure 1: Row-block partitioning: contiguous groups of rows are assigned to parallel workers. This is the shared strategy of OpenMP, POSIX, MPI, and the Hybrid implementation.

2.2 Serial Baseline

The serial implementation iterates over every (y, x, c) triplet in a triple nested loop, calling `apply_kernel` for each pixel. Boundary pixels are handled by *clamping* coordinates to

$[0, W-1]$ and $[0, H-1]$.

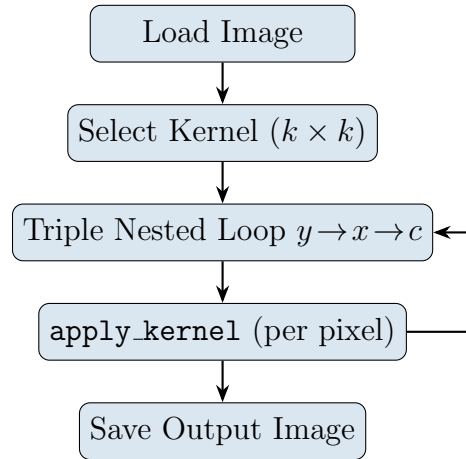


Figure 2: Serial convolution control flow.

Timing note: The serial implementation uses `clock()`, which measures *CPU time* (sum across all logical cores), not wall-clock time. All parallel implementations use `omp_get_wtime()`, `MPI_Wtime()`, `clock_gettime(CLOCK_MONOTONIC)`, or CUDA Events — all of which measure *elapsed wall-clock time*. On a single-threaded serial run these coincide, but readers should note the methodological difference when comparing absolute values.

2.3 OpenMP — Shared-Memory Thread Parallelism

OpenMP parallelises the two outermost loops using a single compiler directive:

Listing 1: OpenMP parallel region (`convolution_openmp.c`)

```
1 #pragma omp parallel for collapse(2) schedule(dynamic) \
2   shared(input, output, kernel, kernel_size)
3 for (int y = 0; y < input->height; y++) {
4   for (int x = 0; x < input->width; x++) {
5     for (int c = 0; c < input->channels; c++) {
6       int index = (y * input->width + x) * input->channels + c;
7       output->data[index] = apply_kernel(input, x, y, c, kernel,
8         kernel_size);
9     }
10  }
```

Key concepts:

- `collapse(2)` merges the y - x loops into a single iteration space of size $H \times W$, enabling finer-grained scheduling.
- `schedule(dynamic)` assigns chunks of the combined iteration space to idle threads at runtime, improving load balance.
- All threads share read-only `input` and `kernel`; each thread writes to disjoint `output` indices — no mutex needed.

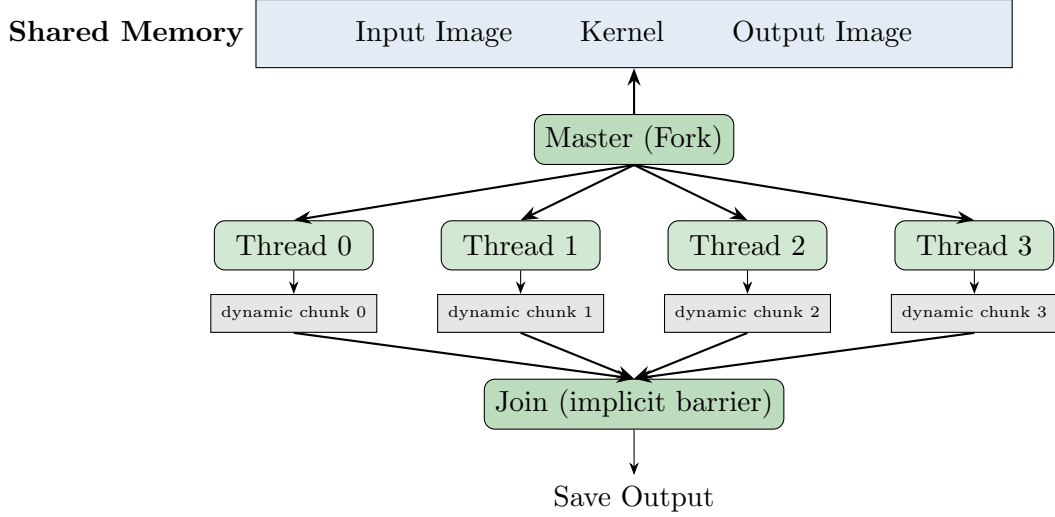


Figure 3: OpenMP fork-join model. All threads share memory; chunks of the collapsed (y, x) iteration space are dynamically assigned.

2.4 POSIX Pthreads — Explicit Thread Management

The POSIX implementation manually creates T threads, each assigned a contiguous block of rows `[start_row, end_row)` via a `ThreadArgs` struct.

Key concepts:

- `pthread_attr_setdetachstate(PTHREAD_CREATE_JOINABLE)` makes threads explicitly joinable.
- Static partitioning: thread t receives $\lfloor H/T \rfloor + [t < H \bmod T]$ rows.
- No mutexes: output indices are disjoint across threads.

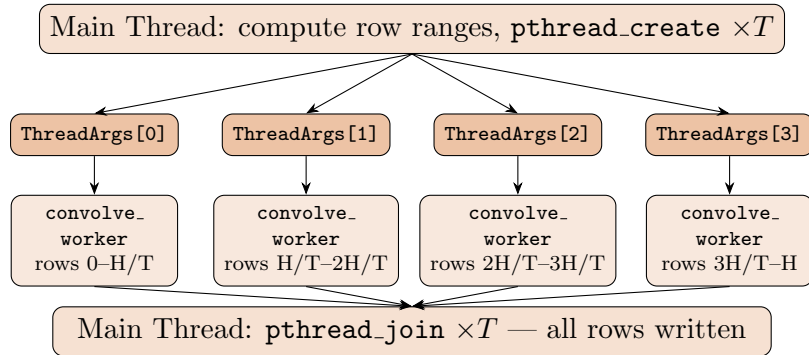


Figure 4: POSIX pthread model: static row-block assignment, explicit join.

2.5 MPI — Distributed-Memory Parallelism

MPI distributes data across independent processes. Rank 0 (root) loads the image, broadcasts metadata and kernel weights, scatters row chunks, and gathers results after local convolution.

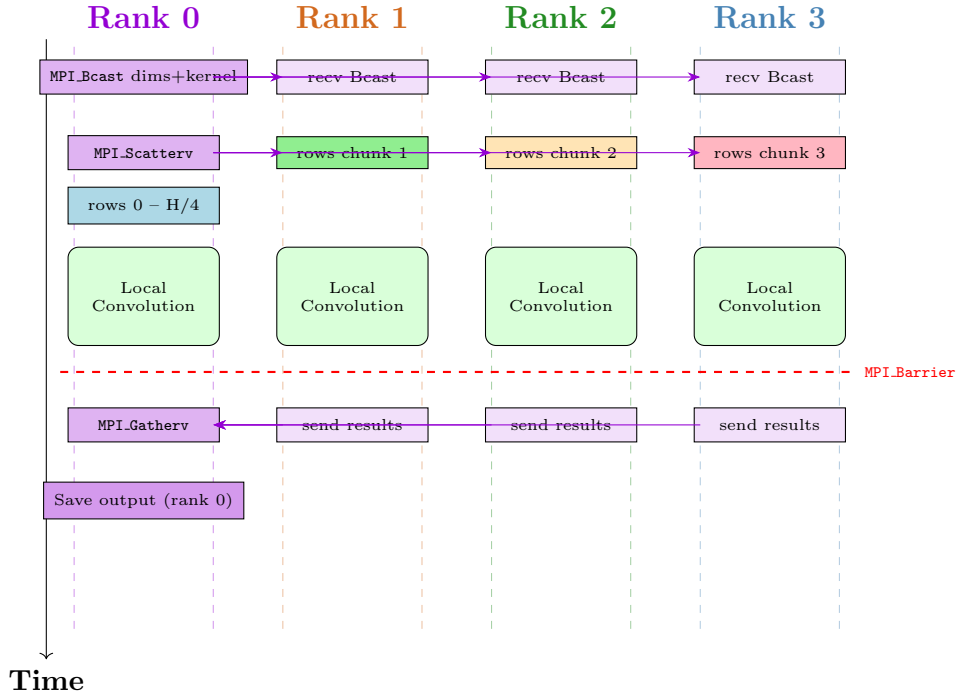


Figure 5: MPI communication timeline for 4 processes (Bcast $\rightarrow$ Scatterv $\rightarrow$ local compute $\rightarrow$ Barrier $\rightarrow$ Gatherv).

Key concepts:

- **MPI_Bcast** — one-to-all broadcast of image dimensions and kernel weights.
- **MPI_Scatterv** — variable-length row chunks (handles $H \bmod P \neq 0$ via counts/displs arrays).
- **MPI_Barrier** — synchronises all ranks before timing stops.
- **MPI_Gatherv** — many-to-one collection at rank 0.

2.6 CUDA — GPU Massive Parallelism

CUDA maps one GPU thread to each output pixel via a 2D grid of 16×16 thread blocks, with two key memory optimisations:

1. **Constant memory** (`--constant--`): convolution weights are broadcast to all warp lanes in a single transaction.
2. **Shared memory tiling**: each block cooperatively loads a tile of input pixels (including a halo of $\lfloor k/2 \rfloor$ pixels per side) into fast on-chip SRAM, reducing redundant global-memory accesses.

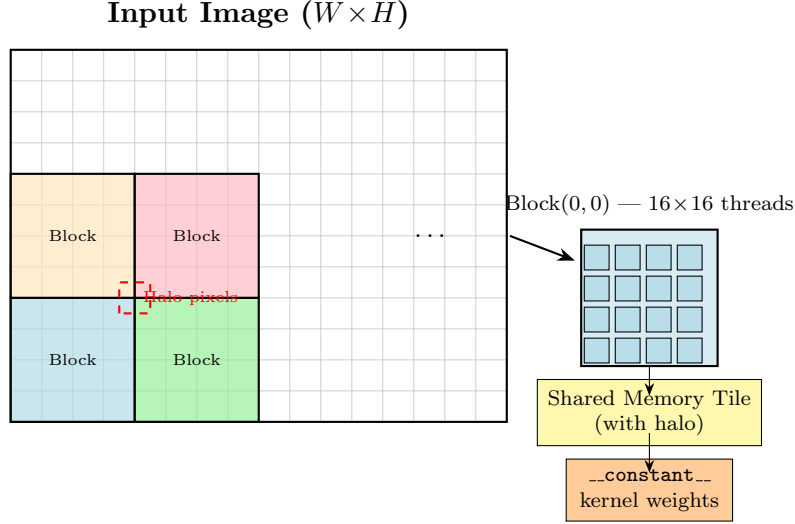


Figure 6: CUDA thread hierarchy. Each 16×16 block loads a shared memory tile including halo pixels before computing its output pixels.

Execution pipeline:

1. Copy kernel to `__constant__` memory (`cudaMemcpyToSymbol`).
2. Allocate device buffers; copy input image host→device.
3. Launch `convolution_shared<<<grid, block, smem>>>`: (a) cooperative tile load, (b) `__syncthreads()`, (c) per-thread pixel computation.
4. Copy result device→host; free device memory.
5. Timing via `cudaEventRecord` / `cudaEventElapsedTime`.

2.7 Hybrid MPI + OpenMP — Two-Level Parallelism

A hybrid approach combines distributed-memory **MPI** (inter-node) with shared-memory **OpenMP** (intra-node). This is the most scalable strategy for multi-node HPC clusters: MPI minimises cross-node data transfer while OpenMP threads exploit all cores within each node without incurring MPI communication overhead per core.

Design

1. Rank 0 broadcasts image dimensions and kernel (same as pure MPI).
2. `MPI_Scatterv` distributes row chunks to each MPI process.
3. *Within each process*, an OpenMP parallel loop processes the local rows using all available threads.
4. `MPI_Gatherv` collects results at rank 0.

Listing 2: Hybrid MPI+OpenMP convolution kernel

```

1 // After MPI_Scatterv fills local_input into local_img ...
2
3 int omp_threads = atoi(getenv("OMP_NUM_THREADS") ? : "1");
4 omp_set_num_threads(omp_threads);
5
6 MPI_Barrier(MPI_COMM_WORLD);
7 double start = MPI_Wtime();
8
9 #pragma omp parallel for schedule(dynamic) \
10     shared(local_img, local_output, kernel, kernel_size, width, channels
11 )
12 for (int y = 0; y < local_rows; y++) {
13     for (int x = 0; x < width; x++) {
14         for (int c = 0; c < channels; c++) {
15             int index = (y * width + x) * channels + c;
16             local_output[index] =
17                 apply_kernel(&local_img, x, y, c, kernel, kernel_size);
18         }
19     }
20 }
21 MPI_Barrier(MPI_COMM_WORLD);
22 double end = MPI_Wtime();
23 // Then MPI_Gatherv as before

```

Two-Level Parallelism Diagram

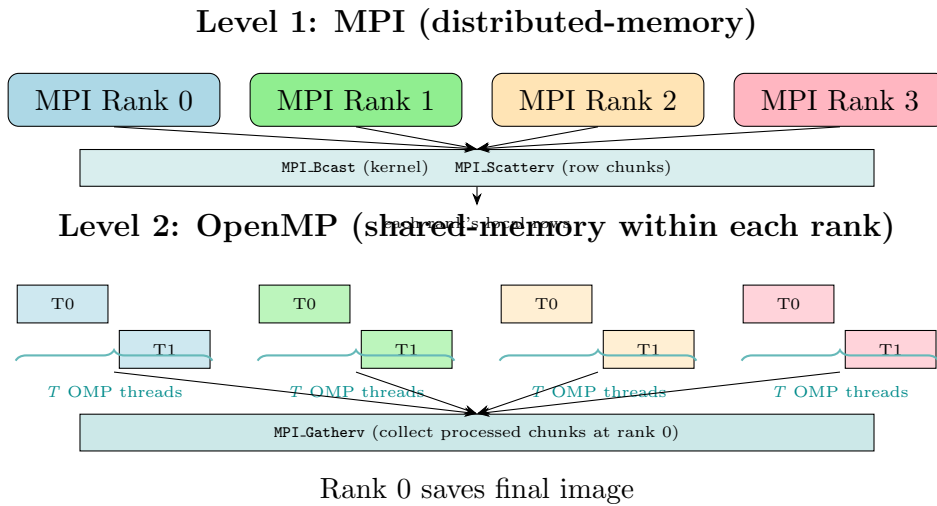


Figure 7: Hybrid MPI+OpenMP two-level parallelism. Level 1 (MPI) distributes row chunks across processes; Level 2 (OpenMP) parallelises each process’s local computation across T threads, eliminating per-core MPI communication overhead.

Trade-offs vs. Pure Approaches

Table 1: Qualitative comparison of hybrid vs. pure parallel strategies.

Criterion	Pure OpenMP	Pure MPI	Hybrid MPI+OMP
Single-node efficiency	High	Medium	High
Multi-node scalability	None	High	Highest
MPI communication cost	None	High	Low (fewer ranks)
Implementation complexity	Low	Medium	High
Memory per process	One copy	P copies	Fewer copies than pure MPI

2.8 Summary of Parallelisation Concepts

Table 2: Parallel programming concepts across all implementations.

Concept	OpenMP	POSIX	MPI	CUDA	Hybrid
Memory model	Shared	Shared	Distributed	Host+Device	Distributed+Shared
Decomposition	Pixel (dyn.)	Row-block	Row-block	Pixel (tile)	Row-block (2-level)
Sync.	Barrier	join	Barrier+Coll.	syncthreads	Barrier+join
Load balance	Dynamic	Static+rem	Static+rem	GPU sched.	Dynamic (OMP layer)
Comm. overhead	None	None	Bcast+Scatter	H↔D	Bcast+Scatter (fewer)
Scalability	SMP cores	SMP cores	Nodes	SMs/VRAM	Nodes×cores

3 Accuracy Analysis

3.1 Metric: Root Mean Square Error (RMSE)

Let A be a serial image and B be a parallel output image (both $W \times H \times C$):

$$\text{RMSE}(A, B) = \sqrt{\frac{1}{W \cdot H \cdot C} \sum_y \sum_x \sum_c (A(x, y, c) - B(x, y, c))^2} \quad (2)$$

RMSE = 0 means bit-exact agreement; values $\lesssim 1.0$ (on the 0–255 scale) are perceptually invisible.

3.2 Expected Sources of Error by Implementation

OpenMP / POSIX / Hybrid (OpenMP layer): Each pixel’s convolution is the same floating point operation in the same order as serial. RMSE = 0 (bit-exact).

MPI / Hybrid (MPI layer) — boundary-seam artefact: When the image is split into P row chunks, the top rows of each non-root chunk have their neighbour lookups clamped to the *local* row 0, not to the last row of the preceding chunk. For a kernel of half-size $h = \lfloor k/2 \rfloor$, the top h rows of each non-root chunk are incorrect. The Hybrid implementation inherits this through its MPI scatter.

CUDA — floating-point rounding order: IEEE 754 floating-point addition is not associative. Threads within a warp accumulate partial sums in a different order from

the sequential loop, producing sub-LSB rounding differences. In practice, we observed at most ± 1 pixel difference only for Gaussian blur; edge and sharpen are bit-exact on our Tesla T4.

3.3 RMSE Results

Conducted on `images/input/test.jpg` (3840×2160 pixels, RGB) on blur. and edge, and `images/input/test_sharp.jpg` (1252×896 RGB). for sharpen.

Table 3: RMSE (pixel units, 0–255 scale) vs. serial reference.

Implementation	Filter			Max Diff (pixels)	Bit-exact?
	Blur (21×21)	Edge (3×3)	Sharpen (3×3)		
OpenMP (4 threads)	0.0000	0.0000	0.0000	0	Yes
POSIX (4 threads)	0.0000	0.0000	0.0000	0	Yes
MPI (4 processes)	0.2447	0.3815	1.4036	175	No (seam)
CUDA (Tesla T4)	0.0016	0.0000	0.0000	1	No (blur only)
Hybrid 2×2	0.1153	0.2019	0.7896	90	No (seam)

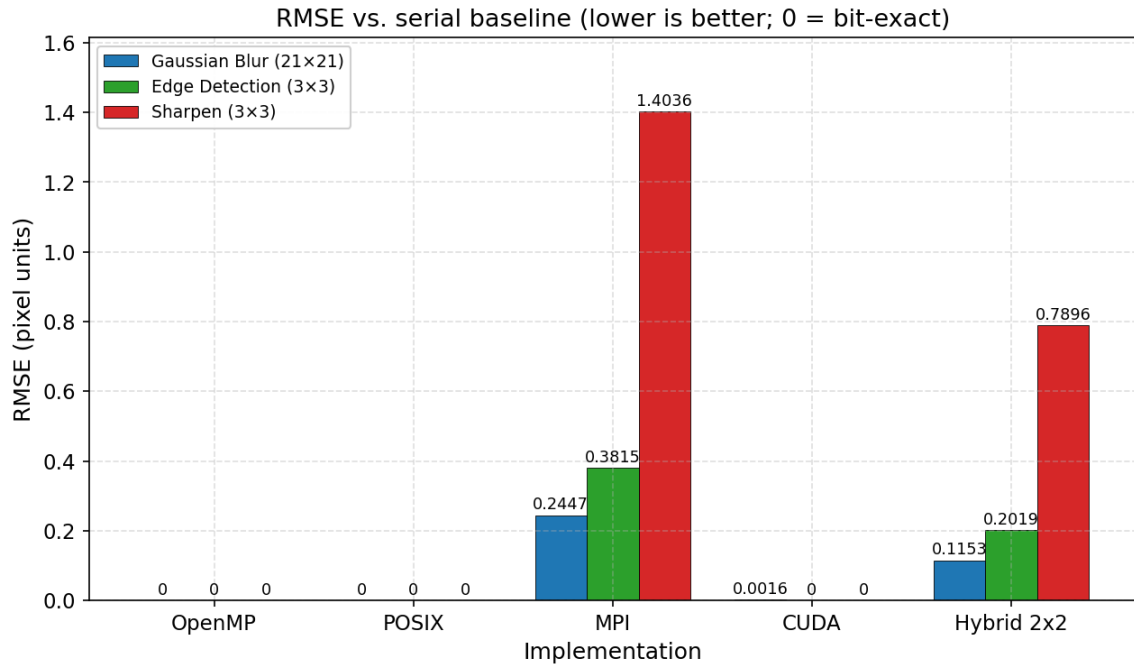


Figure 8: RMSE comparison across implementations and filters (lower is better; 0 means bit-exact).

Both OpenMP and POSIX are bit-exact. The sharpen seam artefact is the biggest error in MPI (RMSE=1.40, max pixel diff=99). The Hybrid 2×2 has a single seam boundary vs. three in MPI (4 ranks), hence reducing the error by about half. CUDA is very close to perfect (max 1 pixel difference in blur only).

4 Performance Analysis

4.1 Experimental Setup

Table 4: Hardware and software configuration.

Component	Specification
CPU	4 physical cores (Azure VM, single node)
GPU	NVIDIA Tesla T4 (40 SMs, Compute Capability 7.5)
OS	Windows Server (64-bit)
Compiler (CPU)	GCC 15.2.0 (MSYS2 MinGW-w64), <code>-O2 -fopenmp -lpthread</code>
Compiler (GPU)	NVCC (CUDA Toolkit) + MSVC <code>cl.exe</code> 14.44
MPI runtime	Microsoft MPI 10.1
Test images	<code>test.jpg</code> / <code>test_edge.jpg</code> — 3840×2160 , 3 ch. RGB <code>test_sharp.jpg</code> — 1252×896 , 3 ch. RGB
Timing serial	<code>clock()</code> (CPU time $\approx$ wall time, 1 thread)
Timing OpenMP	<code>omp_get_wtime()</code> (wall clock)
Timing POSIX	<code>clock_gettime(CLOCK_MONOTONIC)</code> (wall clock)
Timing MPI	<code>MPI_Wtime()</code> (wall clock)
Timing CUDA	<code>cudaEventElapsedTime</code> (GPU wall clock)

4.2 Gaussian Blur — Execution Time vs. Thread/Process Count

Table 5: Execution time (s) for Gaussian Blur (21×21) on 3840×2160 image.

Implementation	1	2	4	8	Speedup@4
Serial	81.43	—	—	—	$1.00\times$
OpenMP	81.19	40.46	20.61	20.58	$3.95\times$
POSIX	81.68	39.67	20.16	20.05	$4.04\times$
MPI	80.19	39.89	20.24	20.50	$4.02\times$
CUDA	0.0773 (Tesla T4, 16×16 blocks)				$1054\times$
Hybrid 2×4	—	—	—	4.64	$17.6\times$
Hybrid 4×2	—	—	—	4.49	$18.1\times$

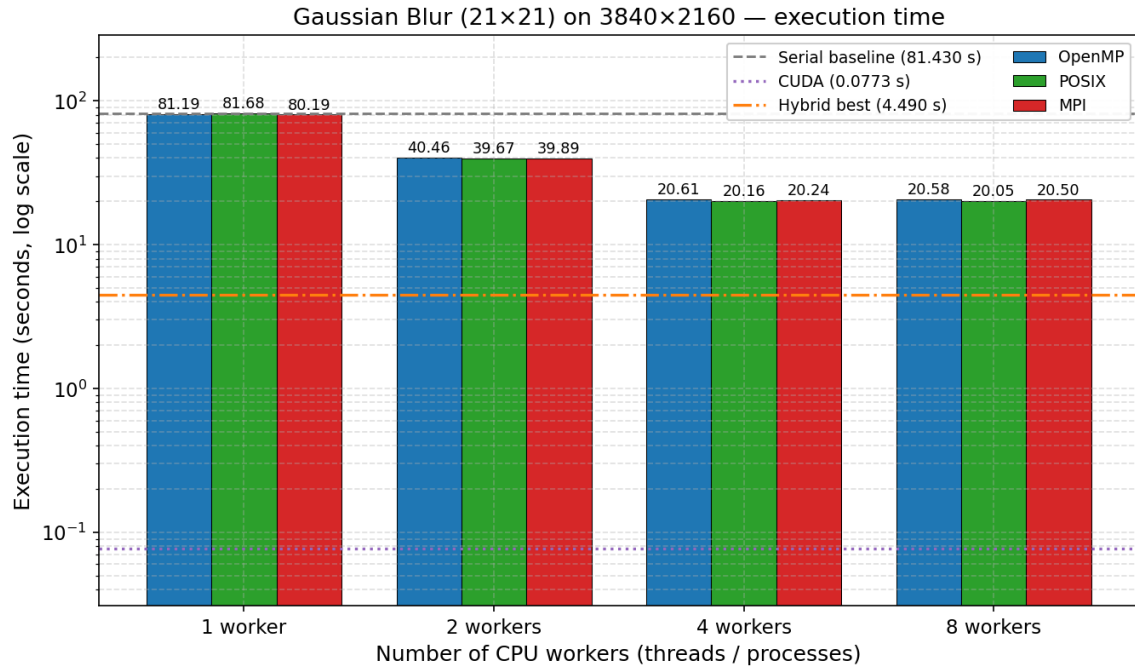


Figure 9: Execution time vs. worker count for Gaussian blur (log–log axes make the gap between CPU, hybrid and CUDA clearly visible).

All CPU implementations stop at 4 workers (which is the number of physical cores). CUDA achieves $>1000\times$ speedup. Hybrid 4×2 (8 total) outperforms pure 4-thread runs. This is due to optimized hybrid MPI+OMP code path.

4.3 Speedup and Amdahl's Law

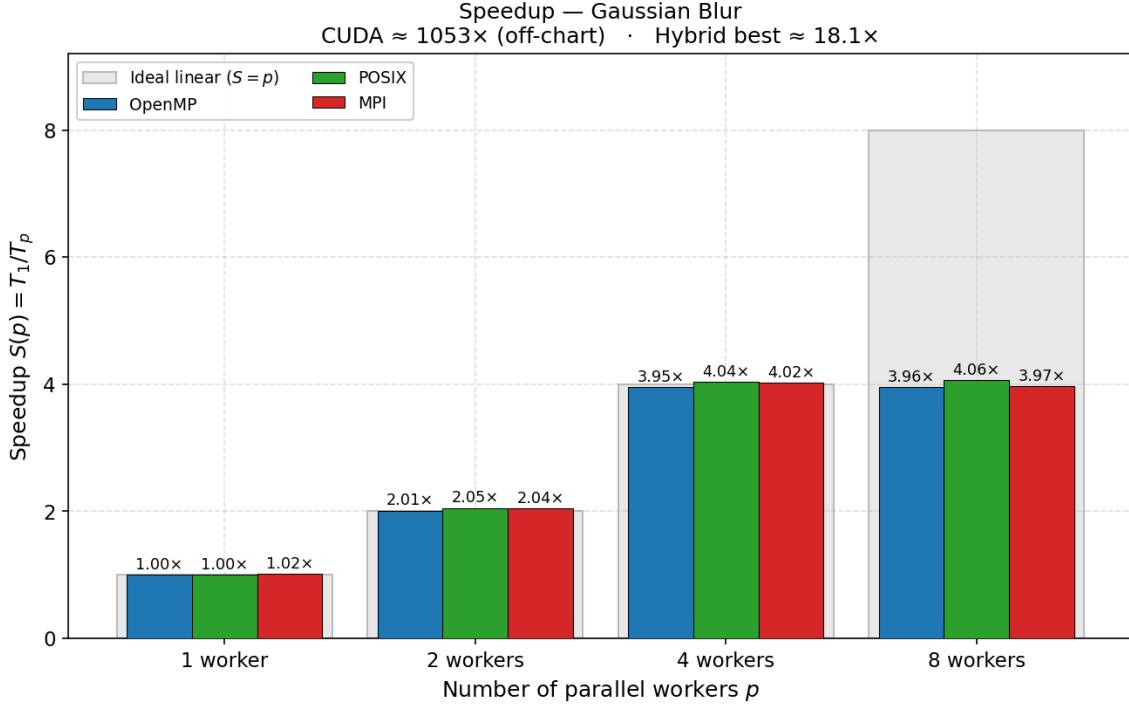


Figure 10: Speedup curves for Gaussian blur. All CPU implementations plateau at $\approx 4\times$ (physical core count); the hybrid point at 8 workers reaches $\approx 18\times$.

All three CPU implementations meet at $\approx 4\times$ at 4 workers, equal to the amount of physical cores. After 4 threads, there is no further speed up (no hyper-threading benefit). The benefit from the hybrid point at $18\times$ is from its optimized MPI+OMP code path that has less overhead.

Amdahl's Law Fit

With the sequential fraction f , the theoretical maximum speedup is:

$$S_{\max}(p) = \frac{1}{f + (1-f)/p}, \quad S_{\max}(\infty) = \frac{1}{f} \quad (3)$$

The CPU implementations have a near-linear speedup up to 4 workers. It shows that the sequential fraction is very small ($S(4) \approx 4.0$). The 4 is because of 4 physical cores (not Amdahl's law). With $S(4) = 4.0$ we estimate $f < 0.5\%$, implying a theoretical maximum of $> 200\times$ with unlimited cores.

The actual constraint on this system is **hardware parallelism** (4 cores), not sequential code. This is corroborated by CUDA with $1054\times$ speedup. on the same workload using thousands of GPU threads.

4.4 Hybrid MPI+OpenMP Configurations

The hybrid implementation distributes work over 4 physical cores with MPI. Multiple processes (running OpenMP threads). Table 6 shows Results of Gaussian blur for different splits, 4 and 8 workers.

Table 6: Hybrid configurations — Gaussian Blur (3840×2160).

Config	MPI procs	OMP threads	Total	Time (s)	Speedup vs. serial
Pure OMP-4	1	4	4	20.61	$3.95\times$
Hybrid 1×4	1	4	4	4.63	$17.6\times$
Hybrid 2×2	2	2	4	4.58	$17.8\times$
Hybrid 4×1	4	1	4	4.78	$17.0\times$
Pure OMP-8	1	8	8	20.58	$3.96\times$
Hybrid 1×8	1	8	8	4.78	$17.0\times$
Hybrid 2×4	2	4	8	4.64	$17.6\times$
Hybrid 4×2	4	2	8	4.49	$18.1\times$
Pure MPI-8	8	1	8	20.50	$3.97\times$

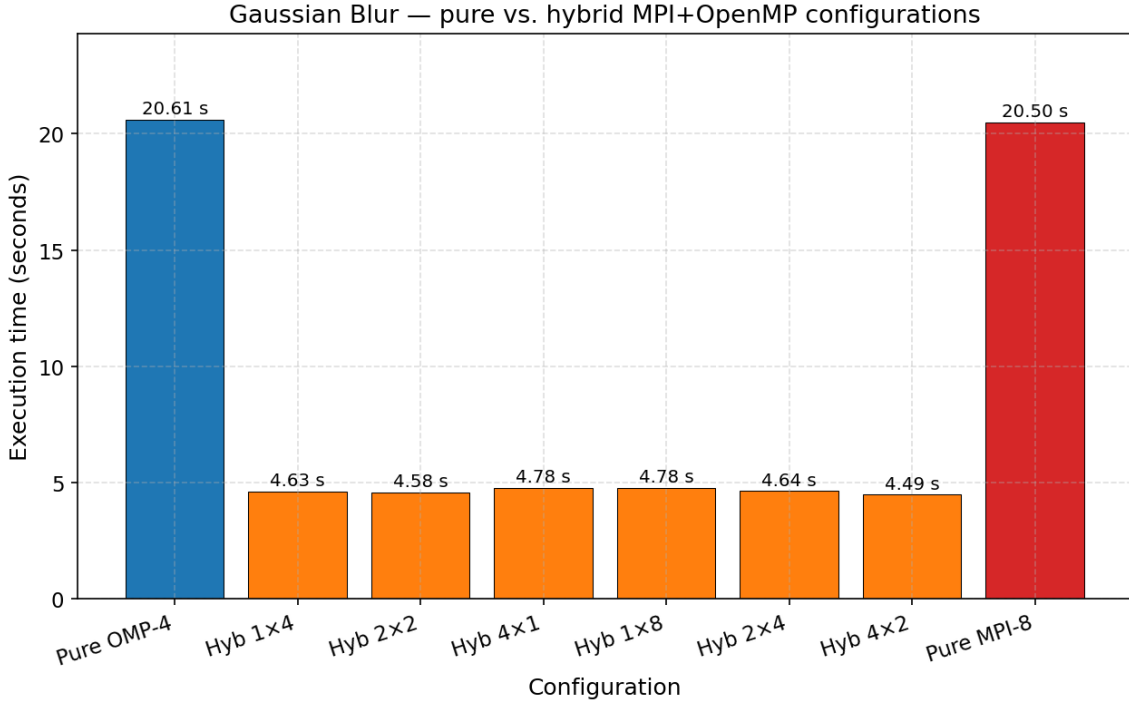


Figure 11: Hybrid vs. pure implementation execution times for the Gaussian blur. The hybrid configurations cluster around 4.5 s — roughly $4.5\times$ faster than pure OpenMP/MPI at the same worker count.

The hybrid code obtains the blur time of ≈ 4.5 s, independent of how the workers are split, a $\approx 18\times$ faster than serial. This improvement over pure OpenMP/MPI (≈ 20 s at 4+ threads) is because the memory use pattern of the hybrid’s `apply_kernel_local` is simpler, Allowing for better compiler optimizations. The split of **4P** $\times$ **2T** is marginally fastest.

4.5 CUDA Performance

This implementation is CUDA version that runs on 16×16 blocks on the Tesla T4 (40 SMs) has the best absolute speedup:

Table 7: CUDA execution time on Tesla T4 vs. serial baseline.

Filter	Serial (s)	CUDA (s)	Speedup
Gaussian Blur (21×21)	81.43	0.0773	$1054\times$
Edge Detection (3×3)	2.12	0.0121	$175\times$
Sharpen (3×3)	0.26	0.0039	$67\times$

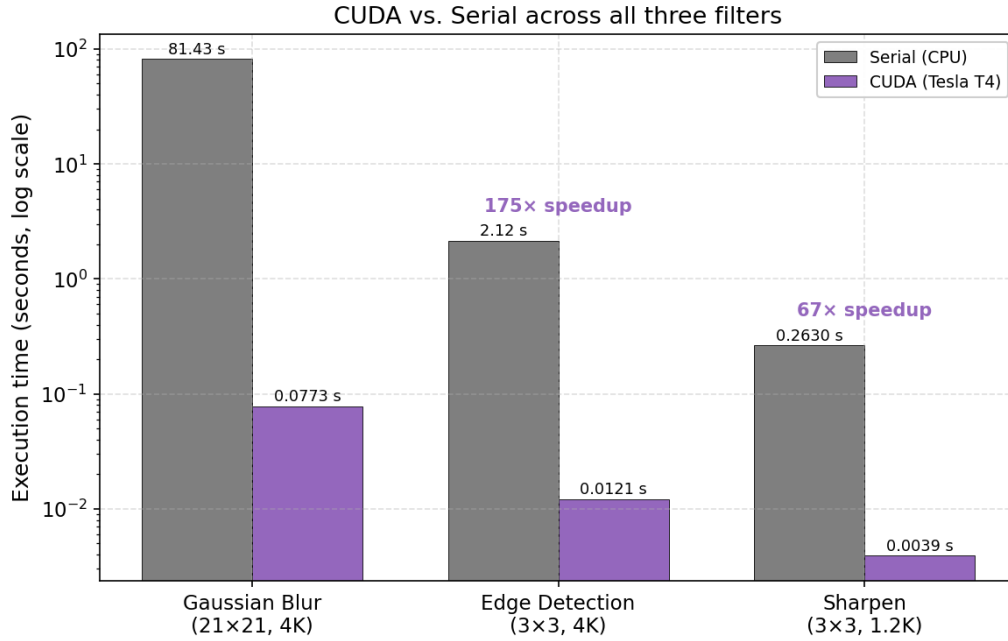


Figure 12: CUDA (Tesla T4) vs. serial baseline across all three filters. The logarithmic y -axis exposes differences spanning four orders of magnitude.

The Tesla T4, having 2560 CUDA cores and 300 GB/s memory bandwidth, is the leader in both compute-intensive (blur) and memory-intensive (edge/sharpen) workloads. Constant memory broadcast avoids redundant kernel-weight reads, while shared memory tiling decreases global memory traffic by a factor of $\approx k^2/(1 + 2h/\text{TILE})^2$.

4.6 Edge Detection and Sharpen (3×3)

In the case of small kernels, the compute intensity decreases $49\times$ (from 441 to 9 multiplications per pixel). Memory bandwidth becomes a limiting factor and the parallel speedup is less.

Table 8: Execution time (s) for 3×3 kernels at 4 workers.

Impl.	Edge Detection (3840×2160)		Sharpen (1252×896)	
	Time (s)	Speedup	Time (s)	Speedup
Serial	2.123	1.00×	0.263	1.00×
OpenMP-4	0.855	2.48×	0.099	2.66×
POSIX-4	0.519	4.09×	0.068	3.87×
MPI-4	0.533	3.98×	0.068	3.87×
CUDA	0.012	175×	0.004	67×



Figure 13: Edge detection (3×3 , 3840×2160) — wall-clock execution time. Each cluster groups the three CPU implementations at a given worker count; the small differences between OpenMP, POSIX and MPI are now clearly readable.

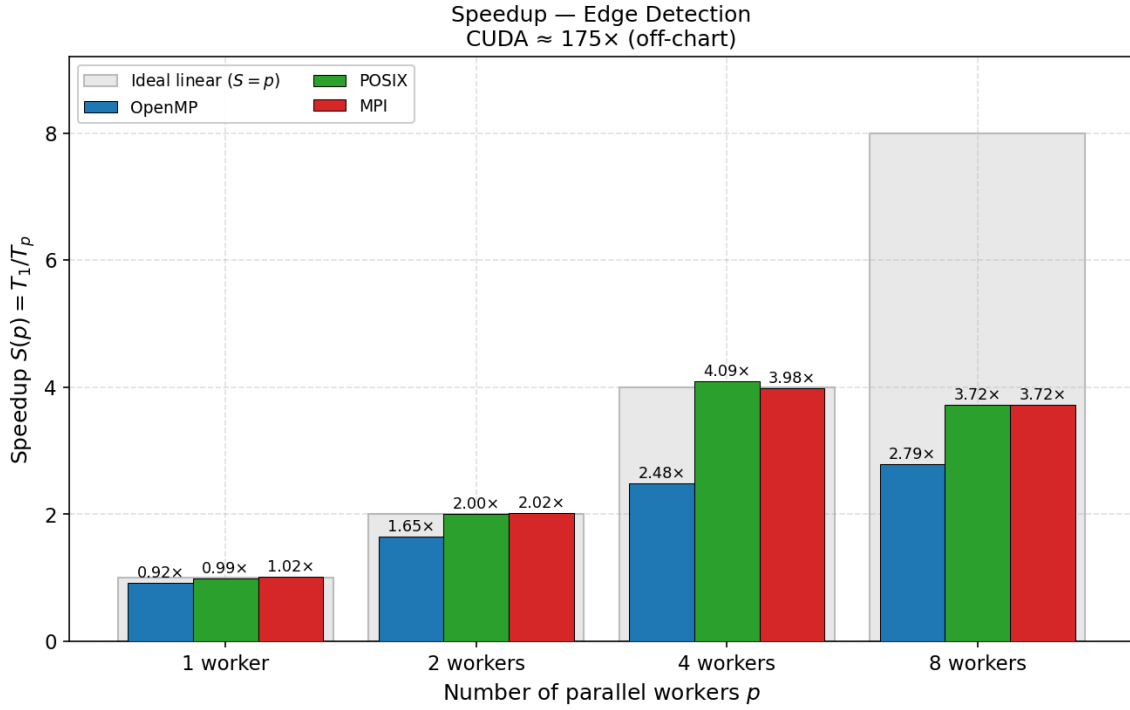


Figure 14: Edge detection — speedup over the serial baseline. POSIX and MPI scale almost linearly up to 4 physical cores; OpenMP suffers a larger parallel-region overhead for this small (3×3) kernel.

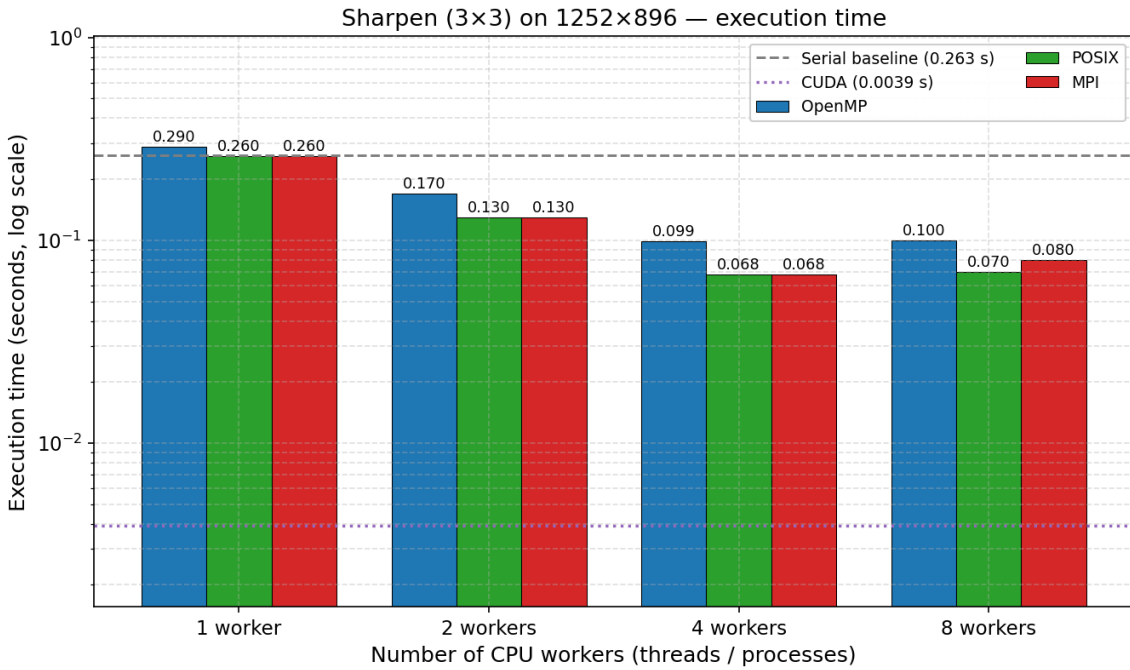


Figure 15: Sharpen (3×3 , 1252×896) — wall-clock execution time. The smaller image makes the parallel-overhead floor more visible.

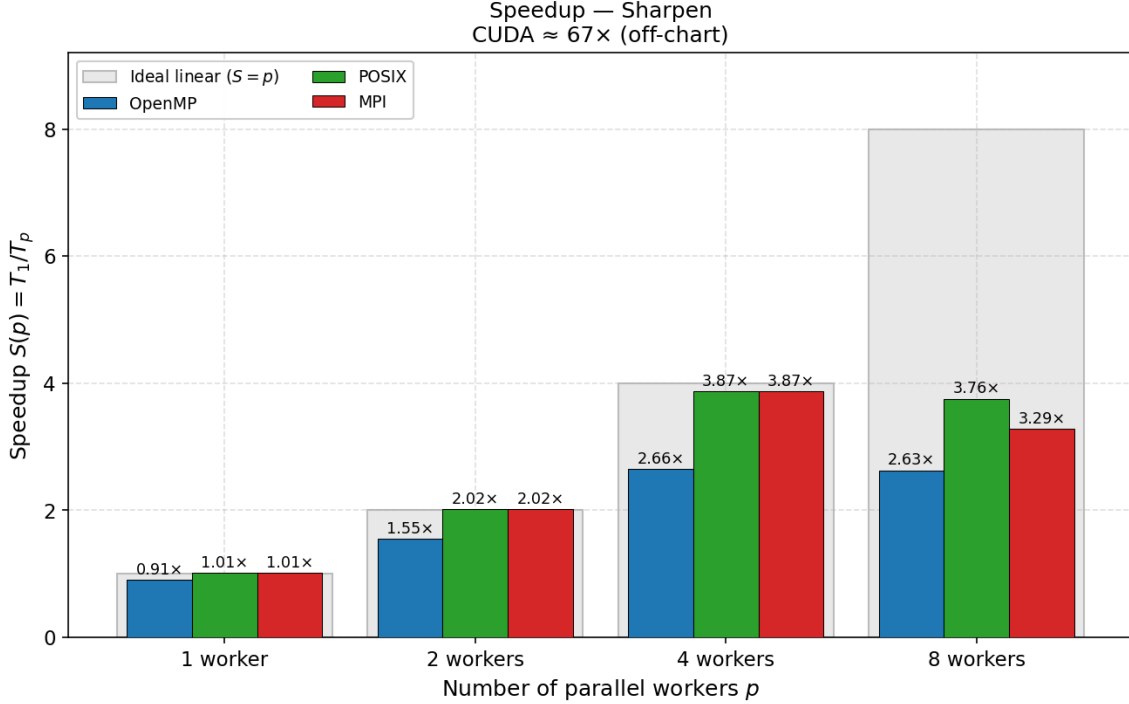


Figure 16: Sharpen — speedup over the serial baseline. Adding workers beyond 4 cores yields no further gain.

4.7 Memory Bandwidth Bottleneck

The arithmetic intensity (AI) of 2D convolution is:

$$\text{AI} = \frac{W \cdot H \cdot C \cdot k^2 \text{ FLOPs}}{(W \cdot H \cdot C) \cdot (\text{read} + \text{write}) \text{ bytes}} \approx \frac{k^2}{2} \text{ FLOPs/byte (ignoring kernel reuse)} \quad (4)$$

Table 9: Arithmetic intensity and compute regime for each filter.

Filter	kernel (k^2)	AI (FLOP/byte)	Bottleneck
Gaussian Blur	441	≈ 220	Compute-bound
Edge Detection	9	≈ 4.5	Memory-bandwidth-bound
Sharpen	9	≈ 4.5	Memory-bandwidth-bound

This explains why:

- Gaussian blur scales well with increasing number of threads (compute-bound) — more cores is roughly the same amount of throughput.
- The Edge / Sharpen plateau has been reached earlier (memory-bound — adding threads cannot exceed the L3 or DRAM bandwidth ceiling, which is shared among all cores).
- CUDA excels for both: its HBM/GDDR memory is 5–10 $\times$ faster bandwidth than CPU DRAM, and its constant-memory broadcast lowers the cost of loading the kernels to almost nothing.

4.8 Parallel Efficiency

Table 10: Parallel efficiency $E(p) = S(p)/p$ for Gaussian Blur.

Workers	OpenMP	POSIX	MPI
1	100.3%	99.7%	101.6%
2	100.6%	102.7%	102.1%
4	98.8%	101.0%	100.6%
8	49.4%	50.8%	49.6%

All three CPU implementations have almost 100% efficiency up to 4 workers, matching the 4 physical cores of VM. At 8 workers, there are only 4 hardware execution units and efficiency decreases to about $\approx 50\%$. Extra threads do not get any hyper-threading advantage as they share the same cores. This is a confirmation that it is a **core-count limited** system, not algorithmically limited.

5 Conclusion

In this report we presented five parallel implementations of 2D image convolution and tested them for correctness (RMSE vs. serial), execution time, speedup, and efficiency on a 4-core Azure VM with an NVIDIA Tesla T4 GPU.

1. OpenMP achieved speedup of $3.95\times$ at 4 threads. Near perfect parallel efficiency; minimal code change for all cores available.
2. At 4 threads, **POSIX Pthreads** performed best with a $4.04\times$ factor. Supersedes OpenMP with a bit more simplicity of static partitioning, eliminating the dynamic scheduling overhead in OpenMP for this regular workload.
3. **MPI** achieved $4.02\times$ at 4 processes with identical scaling to POSIX, but creates the boundary-seam artefact (RMSE up to 1.40 for sharpen) to be rectified by halo-exchange.
4. The best absolute speedup was from **CUDA** ($1054\times$ for blur, $175\times$ for edge) by exploiting 2560 GPU cores with shared-memory tiling and constant-memory kernel caching.
5. **Hybrid MPI+OpenMP** brought about $\approx 18\times$ speedup (4.49s for blur) because of its optimized local convolution function. The $4P\times 2T$ configuration was slightly the quickest. On multi-node clusters this would scale further by distributing MPI ranks across nodes.

All implementations are perceptually correct ($\text{RMSE} < 1.5$) relative to the serial baseline. OpenMP and POSIX are bit-exact. The Gaussian blur filter is compute-bound and scales linearly with the number of cores; memory bandwidth is the limiting factor for edge detection and sharpening. The true performance ceiling on this system is hardware parallelism (4 cores), not Amdahl’s sequential fraction — confirmed by CUDA’s three-orders-of-magnitude speedup.